# Ma che I-Pod e I-Pod

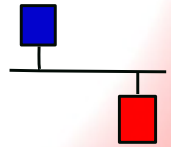



https://gitlab.com/DocSalviS/mp3_player_raspi

© Ing. Stefano Salvi – stefano@salvi.mn.it

# Il problema

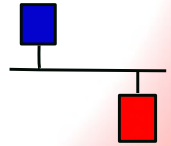
- Ho un **nipotino** di tre anni
- Gli piacciono molto le **canzoni** ed i **video**
- Spesso tutti i parenti usano **Youtube** per trovarli e farglieli vedere
- **Trovare** il video **richiede tempo** ed il bimbo non ha pazienza
- Prima del video viene trasmessa una quantità di **pubblicità**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La soluzione

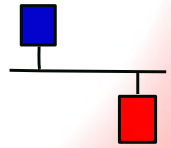
- Ho pensato allora di **realizzare** un '**player MP3**', poi esteso ai **video**
- In questo modo posso **scegliere i video** e le canzoni che voglio
- Ovviamente video e canzoni **non avranno pubblicità**
- Volendo posso anche **aggiungere dei giochi** o degli applicativi didattici (TuxPaint lo adora...)
- Posso creare un'**interfaccia grafica** che consenta **anche al bambino** di scegliere e suonare video e canzoni o aprire i giochi che preferisce



© Ing. Stefano Salvi – stefano@salvi.mn.it

# L’ambiente

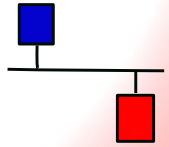
- Una soluzione, **semplice** avrebbe potuto essere quella di creare un’**APP** per cellulare o meglio **tablet** allo scopo
- Questa soluzione, che consente di avere un ambiente sofisticato ed un hardware potente ha però un **inconveniente**
- **Abitua il bambino ad usare il cellulare**, cosa che sarebbe bello (idealmente) evitare.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La soluzione ‘hard’

- Scegliamo allora la **soluzione** ‘hard’: **costruirci tutto in casa**.
- Useremo:
  - Una **scheda con Arm**, dotata di uscita hdmi e audio
  - Un **display 1024x600** dotato di **touch screen**
  - Un piccolo **amplificatore**
  - Un **case** con **due altoparlanti** (da stampare con la stampante 3d)



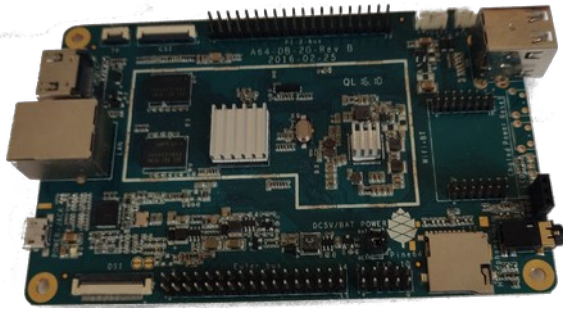
© Ing. Stefano Salvi – stefano@salvi.mn.it

# La scelta dei componenti

- È stato **facile** trovare un **display** adatto
- Anche **l'amplificatore** è stato molto **facile da reperire**
- Per la **scheda CPU**, ho puntato su di una **Pine A64** con **2G di RAM** che avevo comprato molto tempo fa, il cui vantaggio era di avere una buona quantità di RAM.

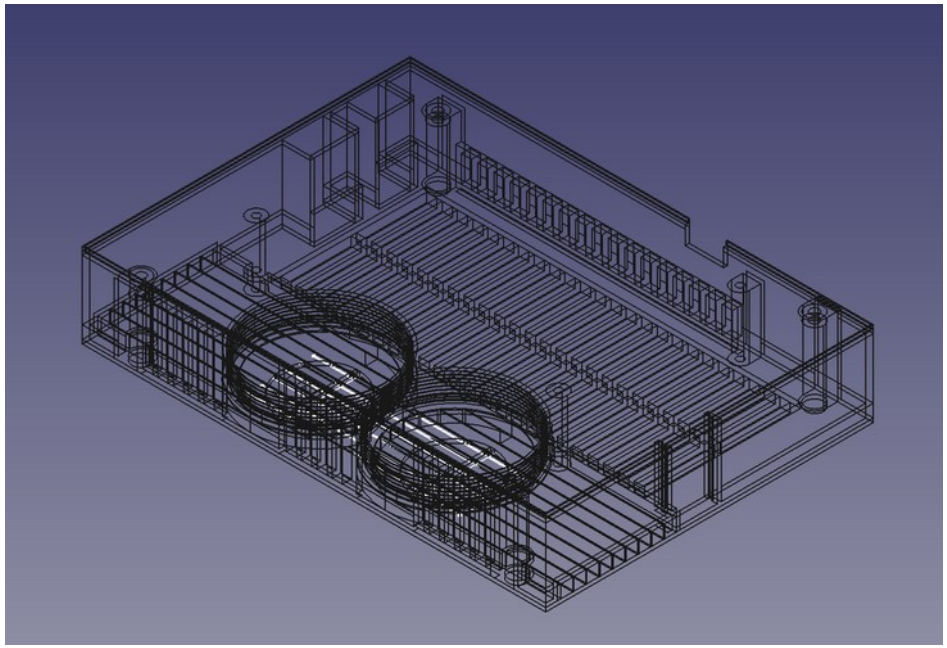






© Ing. Stefano Salvi – stefano@salvi.mn.it

# Il disegno del contenitore

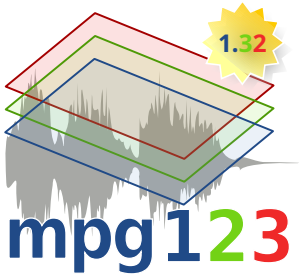


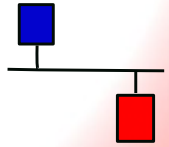
- Il **contenitore** naturalmente **non** era **reperibile sul mercato**
- L’ho quindi **progettato** utilizzando **FreeCad**
- L’ho poi trasformato nel **formato della stampante** (affettato) tramite **Prusa Slicer**
- L’ho infine **mandato in stampa**

© Ing. Stefano Salvi – stefano@salvi.mn.it

# La scelta del software

- Sono partito dall'idea di utilizzare un **player MP3 a riga di comando**, da collegare all'interfaccia grafica
- La scelta cade su **mpg123**
- Questo programma ha tra l'altro una simpatica **modalità '*remote*'** che consente di **comandare** la riproduzione tramite comandi su **standard input** e di **leggere lo stato** tramite **standard output**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# L’ambiente di sviluppo

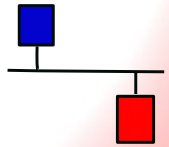
- Per **interfacciarsi facilmente** sia con il **player** che con la **grafica**, la scelta cade su **Python3** e sulla **libreria PyGame**




- **Python3** consente di **controllare** un programma **tramite pipe**, che è **giusto ciò che mi serve** per controllare mpg123
- La libreria **PyGame** consente di **disegnare facilmente** un’interfaccia grafica



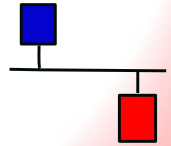

- **PyGame** utilizza la libreria **SDL2** per **collegarsi ai display** sia direttamente tramite **framebuffer** che tramite **X11**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Le prime prove

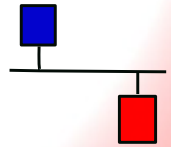
- Da subito il **Pine64** si è rivelato '**debole**' nell'**alimentare il display** tramite l'USB.
- Appena il **carico di CPU aumentava**, il **display** tendeva a **resettarsi**
- Ho quindi deciso di **bypassare X11**.
- Occorreva **interfacciarsi** con il **Framebuffer**
- Per l'**audio** comunque l'interfaccia rimaneva **Alsa**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La scelta del sistema operativo

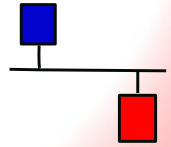
- **Pine** offre una **serie** di **sistemi operativi** (Debian 11, Armbian, DietPi) tra i quali scegliere (mi limito alle derivate di Debian).
- Faccio una **serie di prove** e scopro che con la maggior parte **o non funziona l'audio** oppure **il video da dei problemi**
- Tento **armbian** che fa **funzionare** sia il **Framebuffer** che **l'audio**.
- Rimanevano due problemi:
  - All'avvio il display funziona bene, ma appena si **resetta** cambiava **risoluzione** da **1024x600 ad 800x600**, senza peraltro far sapere al software del cambio di risoluzione
  - Il **touch** del display veniva **interpretato in maniera scorretta**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Mettiamoci una pezza

- Per cercare di far funzionare il tutto:
  - **Scalo l'interfaccia** per portarla ad **800x600** (all'avvio a volte risulta non riempire lo schermo)
  - **Scrivo** una '**driver**' che **legge** direttamente gli **eventi del touch** e li **decodifica**, bypassando la gestione del mouse
- Il **lettore** risulta comunque **instabile**, non usabile
- Decido di **cambiare strada**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Passiamo al Raspberry Pi

- In casa ho **diversi raspberry**, non tutti utilizzati.
- Decido di sceglierne uno.
- Scopro (…) che il **Raspberry 3b**, oltre ad avere la **WiFi** che mi può tornare utile per caricare le canzoni, **possiede** anche un'**uscita audio stereo**.
- Da subito **Raspberry** si è dimostrato **refrattario al Framebuffer**, ma **X11 funziona a meraviglia**
- Il **problema del reset** dello schermo **non si è più presentato**
- Il **touch screen funziona a meraviglia**

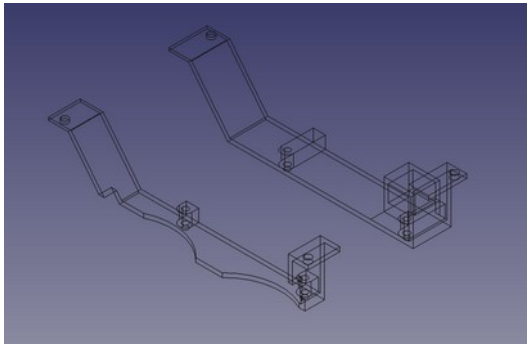


© Ing. Stefano Salvi – stefano@salvi.mn.it

# Modifica del contenitore



- **Decido di utilizzarlo**.
- Per farlo devo **modificare il fondo del contenitore**, per fissare il **Raspberry al posto del Pine A64**.

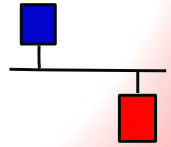


- Data la **complessità** del disegno ed il **tempo di stampa** (circa 5 ore) decido di **non rifarlo completamente**
- Disegno allora una '**culla**' da **fissare** sui **supporti del Pine A64** che tenga il **raspberry in posizione**.

© Ing. Stefano Salvi – stefano@salvi.mn.it

# L’architettura software

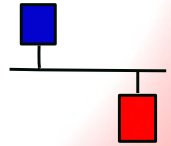
- **PyGame** mette a disposizione una **serie di oggetti** che consentono di **interfacciarsi** al **display**, alla **scheda audio** ed al **mouse/touch screen**
- In primo luogo mette a disposizione un **oggetto `display`** che consente di fare **operazioni sullo schermo**.
- L’oggetto **`display`** contiene un **buffer in memoria** nel quale le **operazioni di disegno scriveranno**
- Quando poi verrà invocato il **metodo `flip()`**, questo buffer verrà **copiato sullo schermo**
- **Creo** una variabile **`screen`** che contiene l’oggetto **`display`** relativo al mio schermo.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Le immagini

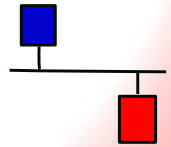
- Per costruire l'interfaccia grafica **non uso primitive di disegno**.
- Tutta l'interfaccia è costruita **copiando immagini sullo schermo**
- Le **immagini** verranno **caricate da file** in memoria di massa (**scheda microSD**) tramite il metodo **`load()`** dell'oggetto **`image`**
- Verranno poi **disegnate nel buffer** invocando **`screen.blit()`**, che **posiziona** ed eventualmente **taglia** o **ridimensiona** l'immagine



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Il mouse

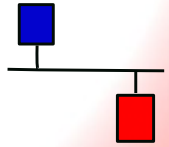
- In **PyGame** esiste una **classe `mouse`** che consente di interagire con mouse, touch screen o altri dispositivi di puntamento.
- È in grado di **fornire eventi**, ma l'architettura ad eventi si integra male con il mio progetto, quindi ho deciso di **leggere posizione e stato dei tasti ad ogni ciclo**.
- Con l'invocazione del metodo **`pygame.mouse.get_pos()`** ottengo le **coordinate assolute** dello schermo, che metto nella variabile **`coords`**
- Con l'invocazione del metodo **`pygame.mouse.get_pressed()`** ottengo lo **stato dei bottoni del mouse** (fino a 5, ma noi useremo sempre solo il primo) che metto in **`buttons`**
- Tengo anche un **conteggio** delle volte che il **mouse viene premuto**, in modo che, se durante la pressione di un bottone il mouse **esce dall'oggetto in cui è stato premuto e viene rilasciato fuori**, oppure se il bottone **entra già premuto**, l'**azione associata non viene effettuata** al rilascio



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Il suono

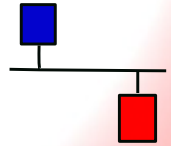
- **PyGame** consente anche di **suonare** dei file **wav** per produrre gli **effetti sonori**.
- A questo scopo **apre il dispositivo audio in maniera esclusiva**.
- In queste condizioni **mpg123 non è in grado di suonare** le canzoni.
- Per **evitare** che questo succeda, occorre ***imbrogliare*** PyGame indirizzando l'audio 'automatico' verso un **dispositivo inesistente**.
- Per fare questo occorre, **prima di inizalizzare PyGame**, impostare la variabile di sistema '**`SDL_AUDIODRIVER`**' con l'istruzione:
  **`os.environ['SDL_AUDIODRIVER'] = 'dsp'`**
- Dove '**`dsp`**' senza percorso non esiste.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La gerarchia degli oggetti

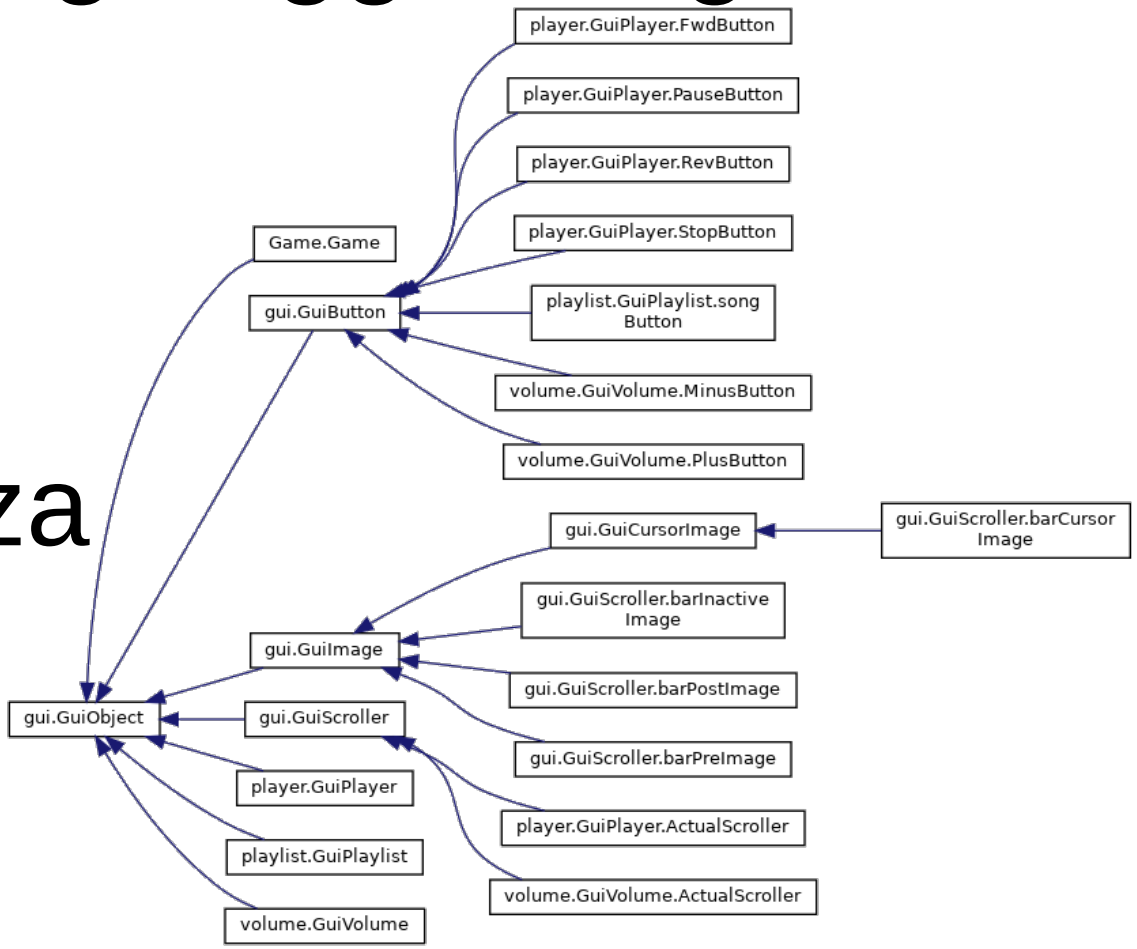
- Per **gestire l'interfaccia** ho creato una **gerarchia di oggetti grafici** che implementano i vari comportamenti dei comuni elementi di interfaccia
  - **Bottoni**
  - **barre di scorrimento**
  - **Pannelli scrollabili**
- Alla **radice** di questa gerarchia troviamo l'oggetto **`GuiObject`**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La gerarchia degli oggetti - grafo

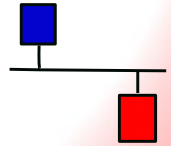
- La **gerarchia** è abbastanza **complicata**
- Deve in sostanza **gestire l'intera interfaccia grafica**
- Ora la vedremo nel dettaglio




© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiObject - struttura

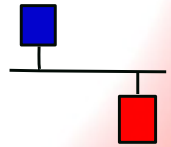
- L'oggetto **GuiObject** è alla radice della gerarchia dell'interfaccia.
- Possiede:
  - Un ***genitore*** (contenutonell'attributo **`parent`**), che è l'**oggetto in cui risiede**
  - Un **elenco**, eventualmente vuoto di *figli* (l'attributo **children**)
- Definisce un'**area sullo schermo**, indicata tramite gli attributi **x**, **y**, **w** ed **h**
- **Valuta** se il **mouse** è **entrato oppure uscito** dall'area dell'oggetto tramite l'attributo **`mouseIn`**
- **Valuta** se il mouse è stato **premuto oppure rilasciato** all'interno dell'oggetto tramite l'attributo **`mousePressed`**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiObject - metodi

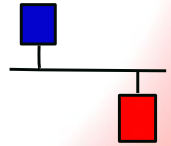
- In **GuiObject** sono definiti, vuoti, **tutti i metodi** che eseguiranno **azioni** relative al **mouse**:
  - **`onMouseDown`**
  - **`onMouseUp`**
  - **`onMouseEnter`**
  - **`onMouseLeave`**
  - **`onMouseMove`**
- **Questi** metodi sono **richiamati** dal metodo **`mouseManage`** che **verifica** se le **coordinate del mouse** sono all'**interno dell'area**, quindi in base al fatto che il mouse sia dentro o fuori, verifica tramite il metodo **`childrenMouseManage`** se per caso è **nell'area di uno dei figli**, altrimenti chiama l'opportuno metodo proprio. Ritorna **`True`** se ha intercettato (e quindi gestito) il mouse, **`False`** altrimenti
- Il metodo **childrenMouseManage** non fa altro che chiamare il metodo **mouseManage dei figli** in ordine inverso, **terminando** al **primo che gestisce** il mouse e ritornando **`True`** se un figlio ha **gestito** il mouse, **`False` altrimenti**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiObject – altri metodi

- Il metodo **`draw`** serve a **disegnare il contenuto** dell'oggetto. Per **`GuiObject`** è vuoto
- Il metodo **`GObjectDraw`** richiama nell'ordine il **`draw`** di **questo oggetto** e poi **tutti quelli dei figli**. Ai **figli** passa le **coordinate del mouse relative**, rispetto all'origine di questo oggetto
- Il metodo **`add`** **aggiunge un figlio** alla lista di figli, impostandone il padre
- Il metodo **`getRelativeCoords`**, metodo di servizio, calcola le **coordinate relative all'oggetto** a partire dalle coordinate assolute, ripercorrendo la catena dei genitori
- È **utile** per implementare le **barre di scorrimento**



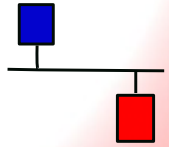
© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiButton

- **`GuiButton`** rappresenta un **bottone** (pulsante) che può essere:
  - **Rilasciato**
  - **Premuto**
  - **Inattivo**



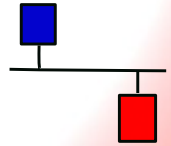
- Se un bottone è **inattivo non reagirà** agli **eventi del mouse**
- Se in un bottone **rilasciato** e viene **premuto** il mouse al suo interno, diventerà **premuto**
- Se il bottone è **premuto**, ma il **mouse esce**, il bottone torna **rilasciato**
- Se il **mouse entra** ed è già **premuto**, il **bottone** diventa premuto solo se il **numero del 'click' corrisponde** a quando **era stato premuto,** quindi non è stato rilasciato e premuto di nuovo mentre era fuori.
- Se un **bottone è attivo** e il mouse viene **rilasciato** al suo interno e il **numero di click** corrisponde a **quando eta stato premuto**, quindi non è stato rilasciato e premuto di nuovo mentre era fuori, **torna rilasciato** e viene invocato il metodo **`onClick()`**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiButton - Metodi

- Quando **creiamo un bottone**, al costruttore dovremo passati, oltre alla **posizione** del bottone, i **nomi** di **tre icone**, una per '**rilasciato**', una per '**premuto**' ed una, eventualmente mancante, per '**inattivo**'
- Il metodo **`draw()` sceglie l'icona** da visualizzare **in base allo stato**
- Viene definito un **metodo `getIcon()`** per **caricare l'icona** dalla **directory** delle icone
- Sono definiti i **metodi `activate()`** e **`inactivate()`** per **cambiare lo stato de bottone**
- Il metodo **`onClick()`** è vuoto
- Le **classi derivate** ridefiniranno il metodo **`onClick()`** con **l'azione** che il **bottone** dovrà **svolgere**.

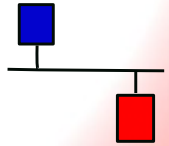


© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiScroller



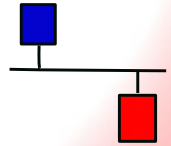
- Questa **classe** implementa un **barra di scorrimento** che può essere sia **verticale** che **orizzontale**.
- **Deriva** direttamente da **`GuiObject`**
- Un **barra di scorrimento** è composta da **tre parti**:
  - Il **cursore** che ne indica la posizione
  - Il **pezzo di barra prima** del cursore (parte già **attiva**)
  - Il **pezzo di barra dopo** il cursore (parte **da attivare**)
- Il **cursore** può essere **trascinato con il mouse** per modificare il valore della barra
- Se si **preme** sulla barra **prima del cursore** (a sinistra o sotto) il valore della barra viene **decrementato** di una certa quantità
- Se si **preme** sulla barra **dopo il cursore** il valore della barra viene **incrementato** di una certa quantità
- Se il **mouse esce** dal cursore **mentre** lo si sta **trascinando**, **continua a muoversi** seguendo il mouse e **nessun altro oggetto** dell'interfaccia **riceve eventi dal mouse**.
- In questo caso **il valore cambierà** solo se il **mouse viene rilasciato all'interno** del cursore



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiImage e GuiCursorImage

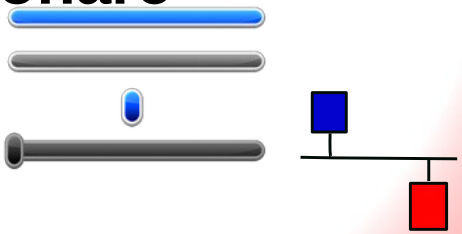
- Per **gestire la barra di scorrimento** ho creato due **classi di appoggio** che sono
- **`GuiImage`** che è **simile ad un bottone**, ma **non cambia aspetto** se viene premuta dal mouse
- **`GuiCursorImage`** che deriva da **`GuiImage`** ma in più **implementa il trascinamento**
- **`GuiCursorImage`** implementa a questo scopo il metodo **`absoluteMouseMove()`** che viene impostato a **livello globale** durante il movimento, per **intercettare il mouse** anche **fuori dall'area del cursore**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiScroller – la logica

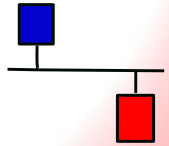
- **`GuiScroller`** richiede **quattro immagini**, incapsulate nelle relative classi:
  - **`barPostImage`** che rappresenta la **parte non suonata** o la parte di volume '**non utilizzata**'. Avrà la parte di **barra scura** e sarà **coperta in parte dalle immagini seguenti**. Deriva da **`GuiImage`**
  - **`barPreImage`** che rappresenta la **parte attiva**, il **valore del volume** oppure la **parte già suonata** di un mp3 o video. Verrà **troncata** modificandone la lunghezza in modo da **terminare sotto il cursore**. Deriva da **`GuiImage`**
  - **`barCursorImage`** l'immagine del **cursore**, la cui **posizione** viene **modificata** in base alla **percentuale della barra**. Deriva da **`GuiCursorImage`**
  - **`barInactiveImage`**, **sopra a tutte**, normalmente invisibile. **Quando viene visualizzata**, **copre** tutta la barra **rendendola inattiva**. Deriva da **`GuiImage`**
- **GuiScroller** si preoccupa, ogni volta che il suo valore viene cambiato, di **riposizionare il cursore** e **ridimensionare `barPreImage`**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiScroller - l'interfaccia

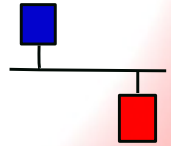
- La **posizione del cursore**, oltre che con il mouse, può venire **modificata** tramite la funzione **`move()`** alla quale viene passata la **nuova percentuale del controllo**
- Possiede tre metodi per gestire le azioni:
  - **`piu()`** che viene richiamata quando si clicca su **`barPostImage`**
  - **`meno()`** che viene richiamata quando si clicca su **`barPreImage`**
  - **`posiziona()`** che viene richiamata quando si trascina **`barCursorImage`**
- Possiede anche i metodi **`activate()`** e **`inactivate()`** come i bottoni



© Ing. Stefano Salvi – stefano@salvi.mn.it

# La funzione mainLoop()

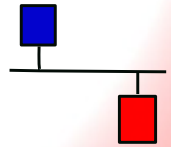
- Termina la **parte comune della GUI** (file gui.py) il main loop **`mainLoop()`**
- A questa funzione viene **passato** un **`GuiObject`** chiamato **`root`** che **copre l'intero schermo** e che **contiene**, come discendenti, tutta **l'interfaccia**.
- La funzione **inizializzerà le variabili d'ambiente** (per 'tenersi per se' l'audio), **pyGame** ed una manciata di **variabili**
- Fatto questo, entra nel **loop principale**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Il loop principale

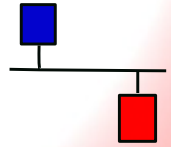
- **Riempie lo schermo** con **il colore dello sfondo**
- Richiama il **ridisegno di `root`** tramite il **metodo `GObjectDraw()`** che lo **propagherà**
- Tramite il **metodo `pygame.display.flip()` invia le modifiche allo schermo**
- **Legge** le **coordinate ed i bottoni del mouse**
- Quindi **valuta** se il **bottone** è stato **premuto o rilasciato dall'ultimo** controllo – genera gli **eventi di pressione e rilascio**
- **Gestisce** il **conteggio** dei click
- **Nel caso** il **mouse** sia stato '**catturato**' da una barra di scorrimento, **invia al metodo `absoluteMouseMove()`** di quell'oggetto l'evento del mouse, **altrimenti** lo invia al **metodo `mouseManage()`** di **`root`**, che lo propagherà
- Tramite il metodo **`clock.tick()`** si **accerta** che il ridisegno **non avvenga più di 20 volte al secondo**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# I controlli complessi

- Ora che mi sono costruito tutto l'armamentario fondamentale che una GUI richiede, mi servono **alcuni componenti complessi** per gestire la mia interfaccia specifica:
  - Il **controllo per il volume**, che contenga una **barra verticale** per regolarlo e **due pulsanti**, uno sopra ed uno sotto per alzarlo ed abbassarlo
  - Il **controllo del player multimediale**, che contenga un **pulsante per terminare** il brano, uno per **metterlo in pausa** o farlo ripartire, uno per **mandarlo indietro**, la **barra della posizione corrente** ed un **bottone** per mandarlo **avanti**
  - Il **pannello** con i **brani da suonare**, che deve poter **scorrere** (senza una scroll bar) e **contenere i pulsanti** che **avvieranno** il player con il relativo **brano** oppure il **gioco**

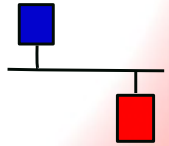


© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiVolume – classi interne

- Nel file volume.py c'è la classe **GuiVolume** che rappresenta il **controllo di volume**
- Contiene **tre elementi**, rappresentati da tre sottoclassi:
  - **`PlusButton`**, derivata da **`GuiButton`**, che alza il volume. Quando cliccato chiama il metodo **`alza()`** del genitore **`GuiVolume`**
  - **`ActualScroller`**, derivata da **`GuiScroller`**, che regola il volume chiamando i metodi **`alza()`**, **`abbassa()`** o **`posiziona()`** del genitore **`GuiVolume`**
  - **`MinusButton`**, derivata da **`GuiButton`**, che abbassa il volume. Quando cliccato chiama il metodo **`abbassa()`** del genitore **`GuiVolume`**

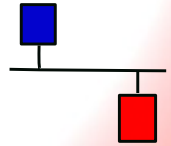




© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiVolume - Funzionamento

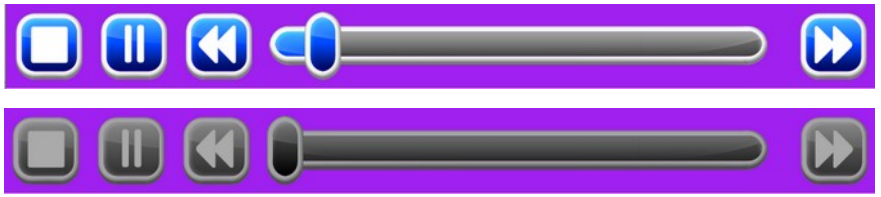
- La classe principale **`GuiVolume`** possiede **tre metodi**, utilizzati dai controlli interni:
  - **`alza()`** che aumenta il volume del 10%
  - **`abassa()`** che abbassa il volume del 10%
  - **`posiziona()`** che pone il volume ad una certa percentuale
- **Tutte e tre** richiamano **`actualMove()`** che **imposta la posizione** della barra e **modifica il volume** richiamando il metodo **`regolaVolume()`**
- Quest'ultimo metodo **scala il valore del volume** nel range **tra 50 e 100** (sotto il 50 il volume è sempre azzerato, nel raspberry) ed esegue il comando "***amixer set PCM Playback <valore> %***" tramite **`subprocesss.run()`** per cambiare il volume tramite **alsa**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlayer

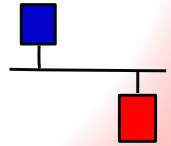
- La **riproduzione dei brani** viene eseguita tramite la classe **GuiPlayer**
- Essa presenta:
  - **Una barra di scorrimento** che indica la **posizione corrente** della riproduzione e può anche modificarla
  - I **bottoni** per andare **avanti** e **indietro** nel brano
  - Un **bottone riproduzione/pausa**
  - Un **bottone** per **terminare** la riproduzione



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlayer – i bottoni

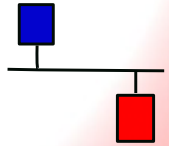
- Sia i **bottoni** che la **barra** sono **classi** che **estendono** rispettivamente **`GuiButton`** e **GuiScroller**
- In particolare:
- **`StopButton`**, **`RevButton`** e **`FwdButton`** si limitano a **richiamare** il corrispondente **metodo** della **classe padre**
- **`PauseButton`** invece **deve cambiare icona** a seconda che il brano sia in **pausa** (il bottone diventa ***play***) o in **riproduzione** (il bottone diventa ***pause***)
- Per ottenere questo il bottone possiede **due set di icone**, una con **l'icona di *pause*** rilasciata e premuta ed una con **l'icona di *play*** rilasciata e premuta
- Possiede tre metodi utilizzati dal padre **`setPause()`**, **`setPlay()`** e **`togglePlay()`** per passare da un set all'altro
- **Al click** richiama il metodo **pause()** del genitore che si occuperà di scegliere il set giusto



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlayer – la classe

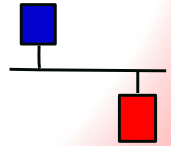
- La classe principale **`GuiPlayer` offre** ai bottoni e alla barra **una serie di metodi**:
  - **`stop()`**
  - **`pause()`**che si occupa anche di commutare il pulsante tra ***riproduci*** e ***pausa***
  - **`indietro()`**
  - **`avanti()`**
  - **`riposiziona()`**
- Queste funzioni **fanno da interfaccia** verso il ***palyer*** vero e proprio
- Abbiamo poi il metodo **`play()`** tramite il quale si può richiedere l'esecuzione di un file



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlayer – il metodo play()

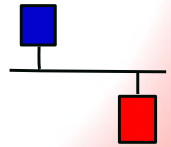
- Per iniziare a riprodurre un file il sistema invoca il metodo **`play()`** di **`GuiPlayer`**
- A questo metodo viene indicato il **percorso del file** e se il file è **audio, video o programma** (gioco)
- A **seconda del tipo** di file, **play** installerà l'**opportuna interfaccia** al **player fisico**.
- L'interfaccia avvierà la riproduzione ed interagirà con **`GuiPlayer`** tramite due metodi:
  - **`setPosition()`** che **riceverà la posizione** nel file e **aggiornerà la barra** di esecuzione
  - **`stop()`** che **segnalerà il termine** della riproduzione e **metterà il player nello stato di inattivo**
- **Se** il player **sta già riproducendo** un file, il metodo **`play()`** **interrompe** l'esecuzione del file corrente e **avvia** la riproduzione del **nuovo contenuto**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlayer – colloquio con i player

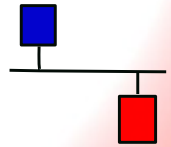
- Quando viene invocato il metodo **`play()`** di **`GuiPlayer`**, se non c'è già qualcosa in riproduzione, a **seconda del tipo** di file (audio, video o programma) crea un ***player*** adeguato, mettendo in riproduzione il file indicato o, nel caso di programma, lo lancia
- Una volta avviato il ***player***, **`GuiPlayer`** può interagire tramite cinque metodi:
  - **`fwd()`** invia al ***player*** il comando di avanzamento di 5 secondi
  - **`rev()`** invia al ***player*** il comando di riavvolgimento di 5 secondi
  - **`pause()`** invia al ***player*** il comando di pausa/riproduzione
  - **`riposiziona()`** che modifica il **punto di riproduzione** (funziona solo per MP3 – per i video il ***player*** non mi consente di farlo)
  - **`stop()`** invia il comando per fermare la riproduzione e chiude il ***player***
- In cambio il ***player*** richiama **periodicamente** il metodo **`SetPosition()`** di **`GuiPlayer`** indicando il **punto di riproduzione** in modo da aggiornare la barra di scorrimento



© Ing. Stefano Salvi – stefano@salvi.mn.it

# mpg123 – il player audio

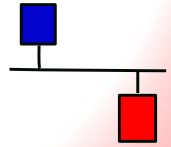
- Il programma ***mpg123*** è un **player mp3 a riga di comando**.
- Ha una interessante **modalità remote** (parametro **`-R`**) nel quale si possono **inviare comandi** tramite ***stdin*** e si **ricevono aggiornamenti** sullo stato tramite ***srdout***
- Il comando viene **lanciato** tramite **`subprocess.Popen()`**
- Per **comunicare** con il comando lanciato devono essere **utilizzati** due '**terminali virtuali**' (**pty**)
- Per **crearli** si usa **`os.openpty()`** che **ritorna due descrittori di file**, uno in **lettura** ed uno in **scrittura** che **comunicano tra di loro**.
- **Uno servirà per inviare comandi** ad ***mpg123*** – su questo canale **scriveremo i comandi**, l'**estremo in lettura** sarà **connesso** a ***stdin*** di ***mpg123***
- L'altro ***pty*** servirà per **ricevere gli aggiornamenti**:
- Il **lato in scrittura** sarà **connesso** ad ***stdout*** di ***mpg123***
- Il **lato in lettura** andrà **gestito da un thread**, creato con **`threading.Tread()`** che stia in ascolto.
- Questo thread **riceverà i dati** e, tramite un **semplice parser**, **rileverà** i **cambiamenti di stato** e la **posizione** di riproduzione inviati da ***mpg123***, richiamando poi il metodo **`setPosition()`** di **`GuiPlayer`**



© Ing. Stefano Salvi – stefano@salvi.mn.it

# ffplay – il player video

- ***ffplay*** è il player video di ***ffmpeg***
- Crea una **finestra** in ***X11*** nella quale **riproduce il video**.
- L'**audio** viene inviato ad ***alsa***
- Invia dei **messaggi di completamento** su ***stderr***
- **Riceve comandi** da **tastiera** tramite **eventi di *X11***
- Per **ricevere lo stato** utilizzo lo **stesso metodo** che per ***mpg123***, ma **associandolo** ad ***stderr*** invece che ad ***stdout***
- Per interagire con il programma devo utilizzare la **libreria `Xlib`**
- Per inviare eventi di tastiera devo utilizzare **`Xlib.ext.xtest.fake_input()`** che genera **`finti eventi di tastiera`**
- Utilizzerò la **tabella `Xlib.XK`** per **recuperare i codici dei tasti**
- Inoltre mi **creerò** un **dizionario** per **ricodificare alcuni dei codici**.



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlaylist

- L'ultimo **componente** necessario è una ***playlist*** dalla quale scegliere.
- Questa sarà un **pannello scrollabile** che conterrà **un bottone per ogni brano, video** o **programma**.
- I **bottoni** conterranno un'**immaginetta** che rappresenta il **particolare contenuto** ed il **nome del file**.
- Se il **file è un video**, nell'angolo in basso a sinistra **comparirà l'icona** di un piccolo **proiettore**.
- Se invece è un **programma o gioco** comparirà l'**icona** di un **controller** da videogioco







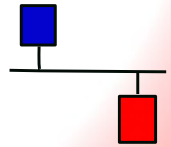



- L'eventuale **file in riproduzione** avrà il **bordo ingrossato**
- Il **contenuto del pannello** potrà essere **trascinato in alto o in basso** per visualizzare tutti i file
- La **playlist** è contenuta in un **oggetto `GuiPlaylist`**, derivata da **`GuiObject`**
- I **bottoni** sono rappresentati da **oggetti `songButton`**, derivati da **`GuiButton`**

© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlaylist – funzionamento pulsanti

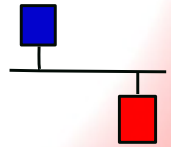
- Quando si **preme un bottone**, il **nome ed il tipo** del file rappresentato verranno **inviati al metodo `play()`** del padre (che è la **`GuiPlaylist`** stessa)
- Il metodo **`play()`** del padre, a sua volta:
  - **Ferma un eventuale brano in riproduzione** usando il metodo **`stop()`** del player (il **`GuiPlayer`**) raggiungibile tramite il padre (il ***root object***)
  - **Avvia** la riproduzione del contenuto scelto usando il metodo **`play()`** del player (il **`GuiPlayer`**) raggiungibile tramite il padre (il ***root object***)
- Per gestire lo scroll, **`GuiPlaylist`** possiede un attributo **`offset`** che indica la coordinata **`y`** del pannello all'interno dell'elenco completo dei bottoni
- Sarà poi **compito dei bottoni** stessi **disegnare il bottone** solo se è almeno in parte visibile e, in questo caso, disegnarne nel posto corretto del pannello solo la parte visibile



© Ing. Stefano Salvi – stefano@salvi.mn.it

# songButton

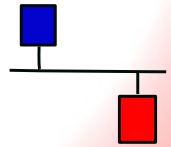
- I **bottoni** sono rappresentati da **oggetti** del tipo **`songButton`**, derivato da **`GuiButton`**
- Questa classe ha due attributi:
  - **song** che **contiene il percorso** della canzone, del video o dell'eseguibile del gioco/programma
  - **`icon`** che **contiene l'immagine** da disegnare nel bottone, già opportunamente scalata
- Il metodo draw(), dopo aver verificato se almeno parte del bottone compare nel pannello,
  1) **Crea una *surface*** della dimensione del bottone, con trasparenza
  2) **Copia dall'array delle rappresentazioni** dei bottoni (intattivo, attivo, disabilitato) l'**immagine del bottone vuoto** nella surface
  3) **Ottiene** un'**immagine con il titolo** della canzone tramite il **metodo render() del font scelto** nel padre
  4) **Copia il titolo** così ottenuto, **centrato ed eventualmente tagliato**, in basso nella *surface*
  5) **Copia** sulla *surface* l'**icona del contenuto**
  6) **Se** il contenuto è un **video**, copia l'**icona del proiettore** in basso a destra sulla surface, se è un **gioco/programma** copia l'**icona del controller**
  7) **Calcola** la **parte** di surface **visibile** e la sua **posizione nello schermo**
  8) **Copia** solo **questa parte sul pannello** (nello schermo)



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlaylist – riempire la lista

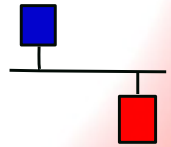
- All'avvio, la **`GuiPlaylist`** fa una scansione della directory **`../mp3`** che contiene, per ogni brano
  - L'icona come file **`.jpeg`**, **`.jpg`** o **`.png`**
  - Il contenuto del brano come file **`.mp3`** per le canzoni, **`.mp4`** o **`.webm`** per i video oppure **directory** per giochi/programmi.
- I **due file/directory** del brano (MP3/MP4/directory ed icona) **condividono il nome** (ad eccezione dell'estensione)
- **Questo nome**, tradotto **in minuscolo**, diventa il **titolo della canzone**
- Per i **contenuti** viene **valutato** se siano **video**, **audio** o **giochi/programmi** in base al **tipo** (se file o directory) ed all'**estensione**
- Per **ogni file**, che sia esso contenuto o icona, **viene chiamato il metodo `addButton()`**, passando il **percorso del file** e indicando se sia contenuto o icona ed **indicando** per i **contenuti** il **tipo** (video gioco/programma o solo audio)



© Ing. Stefano Salvi – stefano@salvi.mn.it

# GuiPlaylist – il metodo addButton()

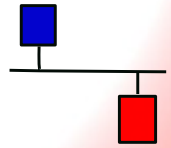
- La funzione **`addButton()`**:
  - **Estrae** dal nome passato (sia esso dell'icona che del contenuto) **il titolo**, eliminando l'estensione e mettendo il nome in minuscolo
  - **Se è stato passato** il percorso dell'**icona**, la **carica e la scala** opportunamente
  - A questo punto **scandisce la lista** dei figli della **`GuiPlaylist`**:
    - **Se trova** un figlio con lo **stesso nome** del contenuto corrente, **aggiorna i dati** di questo **`songButton`** e **termina**
    - **Se arriva in fondo** alla lista, allora il brano non è ancora stato creato, quindi:
      - **Calcola la sua posizione** nella lista scrollabile, spostandosi a destra dall'ultimo bottone e, se esce dal pannello, scendendo di una riga
      - Crea un nuovo **`songButton`** e lo aggiunge alla lista
      - Aggiunge al **`songButton`** le immagini di base dei bottoni, comuni a tutti i bottoni



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Miglioramenti

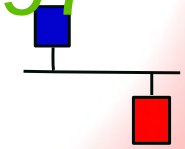
- Lo script (o forse **X11** stesso) occupa una CPU al 100%
- Attualmente il programma ridisegna ad ogni ‘frame’ l’intero schermo e questo sicuramente occupa parecchio la CPU inutilmente
- Di fatto temo che il frame rate previsto di circa 20 frame al secondo non sia rispettato e questo impatta anche sulla responsività dell’interfaccia
- Sarebbe bene evitare i ridisegni nel caso si sappia che non è cambiato niente
- Sarebbe anche bello non ridisegnare sempre l’intero schermo ma limitare il ridisegno ai soli componenti che di volta in volta sono cambiati



© Ing. Stefano Salvi – stefano@salvi.mn.it

# Sitografia

- Specifiche PineA64 https://pine64.org/devices/pine_a64/
- Sistemi operativi per Pine64 https://wiki.pine64.org/wiki/PINE_A64_Software_Releases
- Creare l'autologin di un utente in modalità testuale https://askubuntu.com/questions/175248/how-to-autologin-without-entering-username-and-passwordin-text-mode
- Evitare che PyGame intercetti il dispositivo audio https://stackoverflow.com/questions/31847497/pygame-tries-to-use-alsa



© Ing. Stefano Salvi – stefano@salvi.mn.it